Áp Dụng Clean Architecture & DI Trong Flutter

TRƯỜNG ĐẠI HỌC BÁCH KHOA — KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Chủ đề 04: Báo cáo Tuần 01 — Môn: Lập trình Đa nền tảng

SVTH: Huỳnh Thiện Lộc – **MSSV:** 106230052
GVHD: TS. Nguyễn Duy Nhật Viễn

Đà Nẵng, 2026

Nội dung báo cáo

1. Phần I: Bài Toán Thực Tế & Thiết Kế 3 Tầng
   - Bài toán thực tế & Vấn đề cách làm cũ
   - Áp dụng 3 tầng trong mã nguồn dự án
   - Áp dụng nguyên lý DIP vào mã nguồn
2. Phần II: Hiện Thực Dependency Injection với get_it
   - Tổ chức Container trong injection_container.dart
   - Khởi tạo hệ thống trong main.dart
3. Phần III: Demo Thực Tế & Kết Quả Kiểm Thử
   - Luồng thực thi dữ liệu thực tế (Data Flow)
   - Kiểm thử tự động tầng Domain với Mocktail
   - Tài liệu tham khảo cụ thể

Phân công công việc (WBS)

TT	HỌ VÀ TÊN	NỘI DUNG ĐẢM NHẬN	TỶ LỆ
1	Huỳnh Thiện Lộc (106230052)	Thiết kế 3 tầng Clean Arch, triển khai DI (`get_it`), lập trình BLoC và viết Unit Test Mocktail.	100%

Phần I: Bài Toán Thực Tế & Thiết Kế 3 Tầng

Bài toán thực tế & Vấn đề cách làm cũ

- **Bài toán đặt ra:** Xây dựng ứng dụng đọc 100 bài viết từ REST API (`jsonplaceholder.typicode.com/posts`) và hiển thị mượt mà.

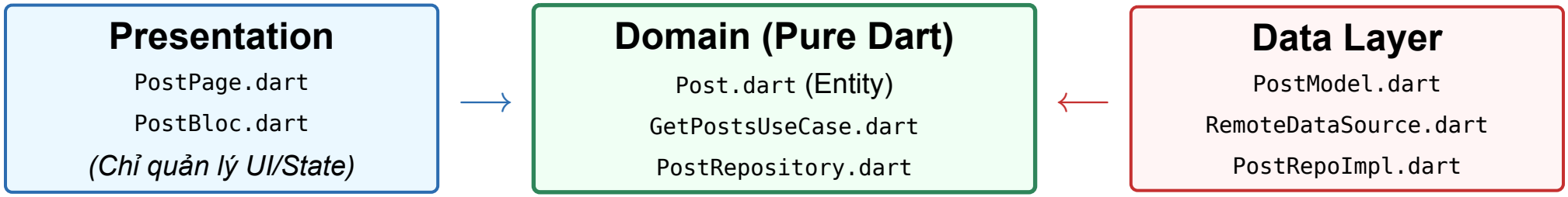
Cách viết cũ (Spaghetti / Naïve)

- Nhét `http.get` trực tiếp trong `initState()` của UI Widget.
- Lẫn lộn parse JSON và logic hiển thị.
- **Hệ quả:** Không thể Unit Test nếu không có mạng; đổi API là phải đập đi sửa lại UI.

Giải pháp: Clean Architecture 3 Tầng

- **Presentation:** `PostPage` + `PostBloc` (UI).
- **Domain:** `Post` + `GetPostsUseCase` (Nghiệp vụ thuần Dart).
- **Data:** `PostRemoteDataSource` + `PostRepoImpl` (Gọi HTTP REST API).

Áp dụng 3 tầng trong mã nguồn dự án



- `PostBloc` chỉ gọi `GetPostsUseCase` và nhận kết quả an toàn bằng `Either<Failure, List<Post>>`.
- Toàn bộ tầng Presentation **không có bất kỳ dòng nào import thư mục `data/`**.

Áp dụng nguyên lý DIP vào mã nguồn

- **Vấn đề cần giải quyết:** Không để Presentation và UseCase phụ thuộc vào `http.Client` hay URL API cụ thể.

Hợp đồng trừu tượng (Interface tại Domain):

```
abstract class PostRepository { Future<Either<Failure, List<Post>>> getPosts(); }
```

Hiện thực tại Data: `class PostRepositoryImpl implements PostRepository { ... }`
Sử dụng tại Presentation: `PostBloc` nhận `GetPostsUseCase(repository: sl())`

- **Ý nghĩa thực tế:** Khi chuyển từ REST API sang Mock test (hoặc Local Cache sau này), code của BLoC và giao diện `PostPage` giữ nguyên 100%, không cần sửa đổi.

Phần II: Hiện Thực Dependency Injection với get_it

Tổ chức Container trong injection_container.dart

- Thay vì class tự khởi tạo phụ thuộc, toàn bộ chuỗi phụ thuộc được nối tại một file duy nhất:

registerLazySingleton (Tạo 1 lần)

- `http.Client` (Client mạng)
- `PostRemoteDataSourceImpl(client: sl())`
- `PostRepositoryImpl(remoteDataSource: sl())`
- `GetPostsUseCase(sl())`

→ *Tiết kiệm bộ nhớ RAM cho các lớp Stateless.*

registerFactory (Tạo mới)

- `PostBloc(getPostsUseCase: sl())`

→ *Tạo instance BLoC mới cho màn hình.*
→ *Đảm bảo giải phóng tự động qua MultiBlocProvider.*
→ *Tránh rò rỉ trạng thái giữa các lần mở.*

Khởi tạo hệ thống trong main.dart

- **Khởi tạo liên kết (`WidgetsFlutterBinding`):**

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized(); // Kết nối Native Engine
  await di.initDependencies();               // Nạp Service Locator
  runApp(const MyApp());
}
```

- **Liên kết phụ thuộc giao diện (`MultiBlocProvider`):**

```
MultiBlocProvider(
  providers: [
    BlocProvider<PostBloc>(create: (_) =>
di.sl<PostBloc>()..add(FetchPostsEvent())),
  ],
  child: const MaterialApp(home: PostPage()),
)
```

Phần III: Demo Thực Tế & Kết Quả Kiểm Thử

Luồng thực thi dữ liệu thực tế (Data Flow)


```
jsonplaceholder.typicode.com/posts ⟶ PostRemoteDataSourceImpl
⟶ PostModel.fromJson ⟶ PostRepositoryImpl ⟶ Post (Entity)
⟶ GetPostsUseCase ⟶ PostBloc ⟶ PostPage (UI Trắng Đen)
```

- **Giao diện tối giản trắng đen:** Tập trung chứng minh xử lý 3 trạng thái:
  1. Đang tải: `CircularProgressIndicator` (Màu đen).
  2. Thành công: `ListView.separated` hiển thị danh sách 100 bài viết (`#ID`, title, body).
  3. Mất mạng: Icon lỗi, thông báo lỗi tiếng Việt và nút **Thử lại** (`FetchPostsEvent`).

Kiểm thử tự động tầng Domain với Mocktail

- **Giải quyết bài toán kiểm thử độc lập:** Sử dụng `mocktail` tạo `MockPostRepository`:

Kịch bản Thành công (Success)

Mock trả về `Right(List<Post>)`

⟶ `UseCase` nhận đúng 2 bài viết mẫu

⟶ `verify(repo.getPosts()).called(1)`

Kịch bản Lỗi máy chủ (Failure)

Mock trả về `Left(ServerFailure)`

⟶ `UseCase` đóng gói lỗi an toàn

⟶ Không crash ứng dụng

- Kết quả chạy lệnh `flutter test`: **5/5 tests pass 100% trong 0.1 giây**, hoàn toàn không cần kết nối mạng Internet.

Tài liệu tham khảo cụ thể

1. **Robert C. Martin (2017)**, *Clean Architecture*, Prentice Hall:
   - **Chương 22 (The Clean Architecture)**: Quy tắc phân chia 3 tầng & Chiều phụ thuộc mã nguồn.
   - **Chương 11 (DIP)**: Nguyên tắc đảo ngược phụ thuộc giữa Module cấp cao và cấp thấp.
2. **Reso Coder (2019)**, *Flutter Clean Architecture Proposal*:
   - Thiết kế `UseCase` (Interactor) và xử lý lỗi Functional bằng kiểu `Either<Failure, T>`.
3. **Felix Angelov**, *Bloc Library Documentation* (https://bloclibrary.dev):
   - Mô hình State Machine trong Flutter và cơ chế vòng đời `MultiBlocProvider`.
4. **Thomas Burkhart**, *get_it Service Locator* (https://pub.dev/packages/get_it):
   - Cơ chế Service Locator & phân biệt vòng đời `registerLazySingleton` vs `registerFactory`.